南開大學
Nankai University

计算机学院

计算机组成原理综合实验实验报告

多周期 CPU 与五级流水线 CPU 改进

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 6 月 22 日

目录

1 实验目的 **3**

2 实验内容 **3**

3 实验原理 **4**
3.1 多周期 CPU 实验原理 4
3.2 五级流水线 CPU 实验原理 4
3.3 实验原理图 4
3.4 数据相关与流水线冲突 4

4 实验设计与实现 **5**
4.1 多周期 CPU 原始实验复现 5
4.2 多周期 CPU Bug 发现与修复 5
4.2.1 `fetch.v` 中 `IF_over` 未复位问题 5
4.2.2 `mem.v` 中 `MEM_valid_r` 未复位问题 6
4.2.3 `multi_cycle_cpu.v` 顶层连接修改 6
4.3 五级流水线 CPU 原始实验复现 6
4.4 五级流水线 CPU Bug 修复与旁路优化 7
4.4.1 `decode.v`：源寄存器分类与旁路优先选择 7
4.4.2 `decode.v`：Wait 判断条件优化 8
4.4.3 `exe.v`：EXE 级旁路输出 8
4.4.4 `mem.v`：Load 等待复位与 MEM 级旁路 8
4.4.5 `wb.v`：WB 级旁路与有效提交保护 9
4.4.6 `multiply.v`：乘法器内部状态复位 10
4.4.7 `pipeline_cpu.v`：旁路总线顶层连接 11
4.5 自定义 MIPS 计算任务设计 11
4.5.1 计算任务选择 11
4.5.2 寄存器分配 12
4.5.3 汇编程序流程 12
4.5.4 机器码与 COE 生成 13
4.5.5 `STARTADDR=0x34` 与前置 NOP 13

5 数据冲突处理方案分析 **13**
5.1 填充空指令方案 14
5.2 Wait 阻塞方案 14
5.3 旁路方案 15
5.4 本实验采用的冲突解决方案 16
5.5 典型数据相关验证 16

6 仿真验证 **17**
6.1 多周期 CPU 原始程序验证 17
6.2 五级流水线 CPU 原始程序验证 19

6.3 五级流水线 CPU 自定义程序验证 20
6.4 数据冲突处理效果验证 22

7 上板验证 **24**
7.1 多周期 CPU 上板验证 24
7.2 五级流水线 CPU 上板验证 25
7.3 自定义程序上板验证 26

8 实验结果分析 **27**
8.1 Bug 修复效果分析 27
8.2 旁路方案与 Wait 阻塞方案对比 28
8.3 自定义计算任务结果分析 28

9 实验总结 **29**

A 关键代码片段 **29**
A.1 多周期 CPU 关键修改 29
A.2 五级流水线 CPU 关键修改 30

B 自定义 MIPS 汇编程序 **31**

C 自定义 COE 文件内容 **31**

1 实验目的

本次实验在复现实验指导手册中多周期 CPU 实验和五级流水线 CPU 实验的基础上，对原始 CPU 设计进行调试、修复和优化。实验的主要目的如下：

1. 理解多周期 CPU 和五级流水线 CPU 的基本结构、数据通路和控制方式。
2. 复现原始 COE 测试程序，验证多周期 CPU 和五级流水线 CPU 的基本功能。
3. 找出原始代码和设计方案中存在的问题，分析 bug 产生原因，并完成修复。
4. 针对五级流水线 CPU 中的数据相关问题，比较填充空指令、流水线 Wait 阻塞和旁路三种解决方案。
5. 仿照单周期 CPU 实验，自行编写 MIPS 汇编计算任务，并放入 COE 文件中交由流水线 CPU 执行。
6. 通过 Vivado 仿真和上板验证，检查修改后的 CPU 是否能够正确执行原始程序和自定义程序。

2 实验内容

本次实验主要包括以下内容：

1. 复现实验指导手册中的多周期 CPU 实验，理解其状态机控制方式、同步 ROM/RAM 访问方式和各阶段数据通路。
2. 复现实验指导手册中的五级流水线 CPU 实验，理解 IF、ID、EXE、MEM、WB 五个流水级之间的控制关系。
3. 分析多周期 CPU 原始代码中存在的复位问题，修复取指阶段和访存阶段等待寄存器初值不确定的问题。
4. 分析五级流水线 CPU 原始代码中存在的数据相关处理不足、部分寄存器状态未复位、有效提交保护不完整等问题，并完成修改。
5. 在五级流水线 CPU 中实现 EXE、MEM、WB 到 ID 阶段的旁路机制，同时保留必要的 Wait 阻塞。
6. 编写自定义 MIPS 汇编程序，完成 1 到 5 的平方和、5!、二者之和以及比较结果的计算，并写入数据存储器。
7. 使用 Vivado 对多周期 CPU 原始程序、流水线 CPU 原始程序和流水线 CPU 自定义程序进行仿真验证。
8. 将程序下载到实验箱进行上板验证，并记录 LCD 或波形中的关键观察结果。

3 实验原理

3.1 多周期 CPU 实验原理

多周期 CPU 将一条指令的执行过程拆分为多个阶段完成，主要包括取指、译码、执行、访存和写回。与单周期 CPU 相比，多周期 CPU 不要求一条指令在一个时钟周期内完成，而是通过状态机控制各个阶段依次执行。

在本实验的多周期 CPU 中，指令存储器和数据存储器均采用同步读方式。当 CPU 给出读地址后，需要等待一个时钟周期才能得到有效数据。因此，取指阶段和 Load 类指令的访存阶段都需要额外的等待控制信号。若这些等待寄存器没有正确复位，仿真初始阶段可能出现不确定值，从而影响 CPU 的正确执行。

3.2 五级流水线 CPU 实验原理

五级流水线 CPU 将指令执行过程划分为 IF、ID、EXE、MEM、WB 五个阶段。多条指令可以在不同流水级中并行执行，从而提高 CPU 的指令吞吐率。

五级流水线虽然提高了执行效率，但也会引入流水线相关问题。其中数据相关最常见，例如后一条指令需要使用前一条指令尚未写回寄存器堆的结果。原始设计主要通过 Wait 阻塞解决数据相关问题，但这种方法会造成较多停顿。本实验在原有 Wait 机制基础上增加旁路，将 EXE、MEM、WB 阶段已经得到的结果提前送回 ID 阶段使用，从而减少不必要的流水线阻塞。

3.3 实验原理图

本次开放性实验的实验原理图采用实验指导手册中的图 9.1(c)，如下所示。

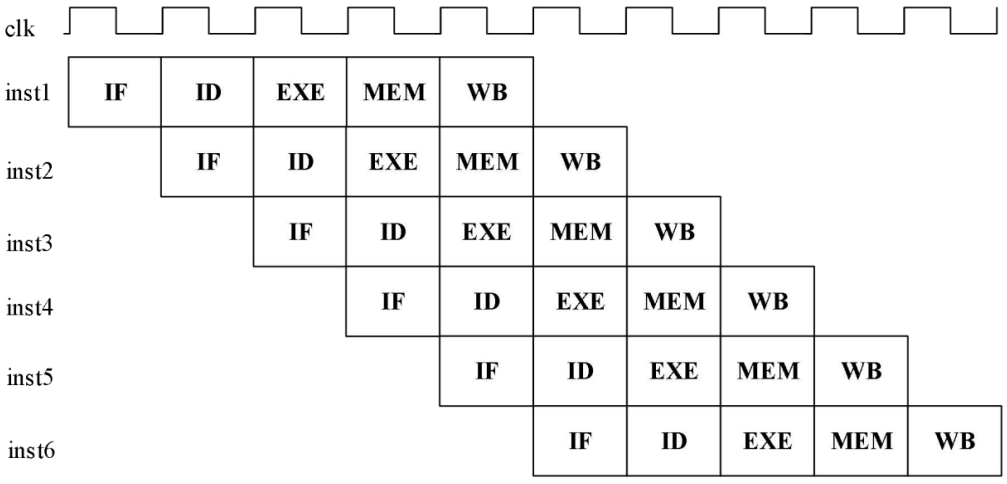


图 3.1: 本次开放性实验原理图

3.4 数据相关与流水线冲突

流水线 CPU 中的数据相关主要指后一条指令需要读取前一条指令的计算结果，但该结果尚未写回寄存器堆。常见解决方法包括：

1. 填充空指令：在相关指令之间手动插入 `nop`，实现简单，但程序效率低。

2. Wait 阻塞：由硬件检测数据相关并暂停流水线，正确性较好，但会增加停顿周期。

3. 旁路：将后级流水线已经产生的结果直接送回前级使用，可以减少等待周期，提高流水线效率。

本实验采用旁路优先、必要时 Wait 阻塞的方案。对于普通 ALU 指令产生的数据相关，优先通过旁路解决；对于 Load 首拍、`mflo` 等结果尚未就绪的情况，仍然使用 Wait 阻塞保证执行正确。

4 实验设计与实现

4.1 多周期 CPU 原始实验复现

多周期 CPU 原始程序按照实验手册要求，使用 `multi_cycle_inst/` 中的 COE 文件建立并初始化指令 ROM 和数据 RAM 的存储器 IP 核。复现时，在 Vivado 中添加多周期 CPU 的 Verilog 源码和仿真文件 `tb.v`，存储器部分不添加 `inst_rom.v` 和 `data_ram.v`，而是由 COE 文件生成的 IP 核提供 ROM/RAM 功能，设置顶层为 `tb` 后运行行为仿真。

仿真过程中主要观察 `IF_pc`、`IF_inst`、`ID_pc`、`EXE_pc`、`MEM_pc`、`WB_pc`、`mem_addr`、`mem_data`、`rf_addr` 和 `rf_data` 等信号。

4.2 多周期 CPU Bug 发现与修复

多周期 CPU 的主要问题出现在同步 ROM/RAM 的等待控制上。由于指令 ROM 和数据 RAM 均为同步读，CPU 发出地址后需要等待一拍才能获得有效数据。因此，取指阶段和 load 访存阶段都需要等待寄存器辅助控制。如果等待寄存器没有复位，仿真开始时可能出现 `X`，导致状态机提前进入下一阶段。

4.2.1 `fetch.v` 中 `IF_over` 未复位问题

在 `fetch.v` 中，`IF_over` 表示取指阶段完成。由于指令 ROM 是同步读，原设计通过将 `IF_valid` 延迟一拍得到 `IF_over`。本次修改加入复位分支：

Listing 1: fetch.v 中 IF_over 复位修复

```
always @(posedge clk)
begin
    // Reset IF_over so the synchronous ROM wait state
    // starts from a known value.
    if (!resetn)
    begin
        IF_over <= 1'b0;
    end
    else
    begin
        IF_over <= IF_valid;
    end
end
```

修改后，复位释放时 `IF_over` 一定从 0 开始。CPU 在取指阶段必须等待一拍后才能进入译码阶段，避免第一条指令因初值不确定而被错误锁存。

4.2.2 mem.v 中 MEM_valid_r 未复位问题

多周期 CPU 的访存阶段中,load 指令需要等待同步数据 RAM 返回数据。原设计使用 MEM_valid_r 表示 load 访存的第二拍，但该寄存器没有复位。本次加入复位逻辑:

Listing 2: mem.v 中 MEM_valid_r 复位修复

```
reg MEM_valid_r;
always @(posedge clk)
begin
    // Reset the one-cycle load wait flag so MEM_over
    // cannot start as X/stale after reset.
    if (!resetn)
    begin
        MEM_valid_r <= 1'b0;
    end
    else
    begin
        MEM_valid_r <= MEM_valid;
    end
end

assign MEM_over = inst_load ? MEM_valid_r : MEM_valid;
```

修改后，load 指令只有在 MEM_valid_r 有效后才认为 MEM 阶段完成，保证写回的数据来自有效的 RAM 读数据。

4.2.3 multi_cycle_cpu.v 顶层连接修改

由于 mem.v 新增了 resetn 输入，因此还需要在顶层 multi_cycle_cpu.v 中接入复位信号:

Listing 3: multi_cycle_cpu.v 中 MEM 模块 resetn 连接

```
mem MEM_module(
    .clk          (clk          ),
    .resetn       (resetn       ),
    .MEM_valid    (MEM_valid    ),
    .EXE_MEM_bus_r(EXE_MEM_bus_r),
    .dm_rdata     (dm_rdata     ),
    .dm_addr      (dm_addr      )
);
```

完成该连接后，访存级内部等待寄存器可以随 CPU 一起复位。

4.3 五级流水线 CPU 原始实验复现

五级流水线 CPU 原始程序按照实验手册要求，使用 pipeline_inst/ 中的 COE 文件建立并初始化指令 ROM 和数据 RAM 的存储器 IP 核。复现时，运行行为仿真并观察 IF_pc、IF_inst、ID_pc、EXE_pc、MEM_pc、WB_pc、cpu_5_valid、mem_addr 和 mem_data 等信号。

原始程序复现用于验证修改后的 CPU 仍能正确执行已有 COE 程序。

4.4 五级流水线 CPU Bug 修复与旁路优化

原始五级流水线 CPU 主要依靠 Wait 阻塞处理数据相关。该方法能保证正确性，但普通 ALU 指令的结果在 EXE 或 MEM 阶段已经产生，如果仍等待到 WB 阶段，会造成不必要停顿。本次在保留必要 Wait 的基础上，实现 EXE、MEM、WB 到 ID 阶段的旁路。

4.4.1 decode.v：源寄存器分类与旁路优先选择

首先在 decode.v 中加入三组旁路输入：

Listing 4: decode.v 中新增旁路输入端口

```
input              EXE_fwd_valid,
input      [  4:0] EXE_fwd_wdest,
input      [ 31:0] EXE_fwd_data,

input              MEM_fwd_valid,
input      [  4:0] MEM_fwd_wdest,
input      [ 31:0] MEM_fwd_data,

input              WB_fwd_valid,
input      [  4:0] WB_fwd_wdest,
input      [ 31:0] WB_fwd_data,
```

随后判断当前指令是否真正读取 rs 或 rt，避免不读取源寄存器的指令产生虚假数据相关：

Listing 5: decode.v 中源寄存器使用分类

```
assign inst_no_rs = inst_J    | inst_JAL  | inst_MTC0
                  | inst_MFC0 | inst_MFHI | inst_MFLO
                  | inst_SLL  | inst_SRL  | inst_SRA
                  | inst_LUI  | inst_SYSCALL | inst_ERET;

assign inst_no_rt = inst_ADDIU | inst_SLTI | inst_SLTIU
                  | inst_BGEZ  | inst_BGTZ | inst_BLEZ
                  | inst_BLTZ  | inst_load | inst_imm_zero
                  | inst_J     | inst_JAL  | inst_JR
                  | inst_JALR  | inst_MFC0 | inst_MFHI
                  | inst_MFLO  | inst_MTHI | inst_MTLO
                  | inst_SYSCALL | inst_ERET;
```

在读取操作数时，按照 EXE、MEM、WB 的优先级选择最新值：

Listing 6: decode.v 中旁路数据优先选择

```
assign rs_data = rs_from_EXE ? EXE_fwd_data :
                 rs_from_MEM ? MEM_fwd_data :
                 rs_from_WB  ? WB_fwd_data  : rs_value;

assign rt_data = rt_from_EXE ? EXE_fwd_data :
                 rt_from_MEM ? MEM_fwd_data :
```

```
7                 rt_from_WB  ? WB_fwd_data  : rt_value;
```

这样，ID 阶段可以优先使用流水线后级已经产生的最新结果，而不是只能等待寄存器堆写回。

4.4.2 decode.v：Wait 判断条件优化

加入旁路后，Wait 条件改为“相关但无法旁路时才阻塞”：

Listing 7: decode.v 中 Wait 判断条件优化

```
assign rs_wait = src_rs & (rs!=5'd0)
              & ( (rs==EXE_wdest) ? ~rs_from_EXE :
                  (rs==MEM_wdest) ? ~rs_from_MEM :
                  (rs==WB_wdest ) ? ~rs_from_WB  : 1'b0 );

assign rt_wait = src_rt & (rt!=5'd0)
              & ( (rt==EXE_wdest) ? ~rt_from_EXE :
                  (rt==MEM_wdest) ? ~rt_from_MEM :
                  (rt==WB_wdest ) ? ~rt_from_WB  : 1'b0 );

assign ID_over = ID_valid & ~rs_wait & ~rt_wait
               & (~inst_jbr | IF_over);
```

因此，普通 ALU 相关可以通过旁路解决；load 首拍、mflo、mfhi、mfc0 等结果尚未就绪的情况仍然通过 Wait 阻塞保证正确性。

4.4.3 exe.v：EXE 级旁路输出

EXE 阶段中，普通 ALU 指令已经得到计算结果，可以输出旁路数据；但 load 和特殊寄存器读取指令在 EXE 阶段尚未得到最终写回值，不能旁路：

Listing 8: exe.v 中 EXE 级旁路输出

```
assign EXE_fwd_valid = EXE_valid & EXE_over & rf_wen
                     & (rf_wdest!=5'd0)
                     & ~mem_control[3] & ~mfhi & ~mflo & ~mfc0;

assign EXE_fwd_wdest = EXE_wdest;
assign EXE_fwd_data  = exe_result;
```

其中 mem_control[3] 用于排除 load 指令，mfhi、mflo、mfc0 用于排除特殊结果尚未就绪的情况。

4.4.4 mem.v：Load 等待复位与 MEM 级旁路

流水线 MEM 阶段同样修复了 load 等待寄存器未复位的问题：

Listing 9: 流水线 mem.v 中 Load 等待寄存器复位

```
reg MEM_valid_r;
always @(posedge clk)
```

```
begin
    // Reset prevents a stale load wait flag from
    // completing a new load too early.
    if (!resetn || MEM_allow_in)
    begin
        MEM_valid_r <= 1'b0;
    end
    else
    begin
        MEM_valid_r <= MEM_valid;
    end
end

assign MEM_over = inst_load ? MEM_valid_r : MEM_valid;
```

当 MEM 阶段已经得到最终写回数据后，产生 MEM 级旁路：

Listing 10: mem.v 中 MEM 级旁路输出

```
assign MEM_fwd_valid = MEM_valid & MEM_over & rf_wen
                     & (rf_wdest!=5'd0)
                     & ~mfhi & ~mflo & ~mfc0;

assign MEM_fwd_wdest = MEM_wdest;
assign MEM_fwd_data  = mem_result;
```

4.4.5 wb.v：WB 级旁路与有效提交保护

WB 阶段输出的旁路数据应与本周期真正写入寄存器堆的数据一致：

Listing 11: wb.v 中 WB 级旁路输出

```
assign rf_wen   = wen & WB_over;
assign rf_wdest = wdest;
assign rf_wdata = mfhi ? hi :
                  mflo ? lo :
                  mfc0 ? cp0r_rdata : mem_result;

assign WB_fwd_valid = rf_wen & (rf_wdest!=5'd0);
assign WB_fwd_wdest = rf_wdest;
assign WB_fwd_data  = rf_wdata;
```

同时，HI/LO 属于体系结构状态，只能由真正到达 WB 阶段并提交的指令修改：

Listing 12: wb.v 中 HI/LO 有效提交保护

```
if (!resetn)
    hi <= 32'd0;
else if (WB_over && hi_write)
    hi <= mem_result;

```

```
if (!resetn)
    lo <= 32'd0;
else if (WB_over && lo_write)
    lo <= lo_result;
```

CP0 相关状态也使用 WB_over 约束更新：

Listing 13: wb.v 中 CP0 有效提交保护

```
assign status_wen = WB_over & mtc0
                  & (cp0r_addr=={5'd12,3'd0});

assign epc_wen = WB_over & mtc0
               & (cp0r_addr=={5'd14,3'd0});

assign cancel = (syscall | eret) & WB_over;
```

4.4.6 multiply.v：乘法器内部状态复位

自定义程序使用 mult 和 mflo 指令，因此乘法器内部状态必须可靠复位。修改中对 multiplicand、multiplier、product_temp 和 product_sign 加入复位：

Listing 14: multiply.v 中乘法器内部状态复位

```
if (!resetn)
    multiplicand <= 64'd0;

if (!resetn)
    multiplier <= 32'd0;

if (!resetn)
    product_temp <= 64'd0;

if (!resetn)
    product_sign <= 1'b0;
```

同时在 exe.v 中实例化乘法器时接入 resetn：

Listing 15: exe.v 中乘法器 resetn 连接

```
multiply multiply_module (
    .clk       (clk       ),
    .resetn    (resetn    ),
    .mult_begin(mult_begin),
    .mult_op1  (alu_operand1),
    .mult_op2  (alu_operand2),
    .product   (product),
    .mult_end  (mult_end)
);
```

4.4.7 pipeline_cpu.v：旁路总线顶层连接

最后在顶层 pipeline_cpu.v 中声明三组旁路总线：

Listing 16: pipeline_cpu.v 中旁路总线声明

```
wire        EXE_fwd_valid;
wire [ 4:0] EXE_fwd_wdest;
wire [31:0] EXE_fwd_data;

wire        MEM_fwd_valid;
wire [ 4:0] MEM_fwd_wdest;
wire [31:0] MEM_fwd_data;

wire        WB_fwd_valid;
wire [ 4:0] WB_fwd_wdest;
wire [31:0] WB_fwd_data;
```

并将其连接到 decode 模块：

Listing 17: pipeline_cpu.v 中 decode 模块旁路端口连接

```
.EXE_fwd_valid(EXE_fwd_valid),
.EXE_fwd_wdest(EXE_fwd_wdest),
.EXE_fwd_data (EXE_fwd_data ),

.MEM_fwd_valid(MEM_fwd_valid),
.MEM_fwd_wdest(MEM_fwd_wdest),
.MEM_fwd_data (MEM_fwd_data ),

.WB_fwd_valid (WB_fwd_valid ),
.WB_fwd_wdest (WB_fwd_wdest ),
.WB_fwd_data  (WB_fwd_data  ),
```

至此，EXE/MEM/WB 到 ID 阶段的旁路网络建立完成。

4.5 自定义 MIPS 计算任务设计

4.5.1 计算任务选择

本次自定义程序计算 1 到 5 的平方和、5!、二者之和以及比较结果，并写入数据 RAM。预期结果如下：

- Mem[0x80] = 0x00000037，即 $1^2+2^2+3^2+4^2+5^2=55$。
- Mem[0x84] = 0x00000078，即 $5!=120$。
- Mem[0x88] = 0x000000AF，即 $55+120=175$。
- Mem[0x8C] = 0x00000001，即 $55<120$。

4.5.2 寄存器分配

寄存器	含义	用途
$8	n	循环上限 5
$9	i	循环变量
$10	sum	平方和累计值
$11	fact	阶乘累计值
$12	square	当前 i^2
$13	done	循环结束判断
$14	total	平方和与阶乘之和
$15	cmp	比较结果
$16	base	数据 RAM 写入基地址

4.5.3 汇编程序流程

Listing 18: 自定义 MIPS 汇编程序

```
addiu $8,  $0, 5       # n = 5
addiu $9,  $0, 1       # i = 1
addiu $10, $0, 0       # sum = 0
addiu $11, $0, 1       # fact = 1
addiu $16, $0, 0x80    # base = 0x80

loop:
mult  $9, $9
mflo  $12
addu  $10, $10, $12

mult  $11, $9
mflo  $11

addiu $9, $9, 1
slt   $13, $8, $9
beq   $13, $0, loop
nop

sw    $10, 0($16)
sw    $11, 4($16)
addu  $14, $10, $11
sw    $14, 8($16)
slt   $15, $10, $11
sw    $15, 12($16)

end:
j end
nop
```

其中 `mflo -> addu` 会产生结果尚未就绪的数据相关，需要 Wait；`addiu -> slt` 和 `slt -> beq`

可以通过旁路减少停顿。

4.5.4 机器码与 COE 生成

自定义汇编程序保存在 `pipeline_inst/custom_sum_factorial.asm` 中，并生成以下文件：

- `pipeline_inst/inst_pipeline_custom.coe`

其中，`inst_pipeline_custom.coe` 用于建立并初始化自定义程序对应的指令 ROM IP 核。主要机器码如下：

Listing 19: 自定义程序主要机器码

```
24080005
24090001
240A0000
240B0001
24100080
01290018
00006012
014C5021
01690018
00005812
25290001
0109682A
11A0FFF8
00000000
AE0A0000
AE0B0004
014B7021
AE0E0008
014B782A
AE0F000C
08000021
00000000
```

4.5.5 `STARTADDR=0x34` 与前置 NOP

五级流水线 CPU 的取指起始地址为 `0x34`。因此自定义 COE 初始化内容前 13 条指令需要填充为 `nop`，使第一条有效指令位于地址 `0x34`。

如果不填充前置 `nop`，CPU 从 `0x34` 开始取指时会跳过程序开头的初始化指令，导致最终计算结果错误。

5 数据冲突处理方案分析

五级流水线 CPU 中，指令被划分为 IF、ID、EXE、MEM、WB 五个阶段并行执行。由于后一条指令可能在 ID 阶段就需要读取前一条指令尚未写回的结果，因此会产生数据相关。本实验重点处

理 RAW（Read After Write）数据相关，即后一条指令读取的寄存器正好是前一条或前几条指令将要写回的寄存器。

以自定义程序中的部分指令为例：

Listing 20: 自定义程序中的典型数据相关

```
mult  $9, $9
mflo  $12
addu  $10, $10, $12

addiu $9, $9, 1
slt   $13, $8, $9
beq   $13, $0, loop
```

其中 `mflo -> addu`、`addiu -> slt`、`slt -> beq` 都存在数据相关。针对这些情况，可以采用填充空指令、Wait 阻塞和旁路三种方案。

5.1 填充空指令方案

填充空指令方案是在存在数据相关的两条指令之间手动插入若干条 `nop`，等待前一条指令写回寄存器后，后一条指令再读取数据。例如：

Listing 21: 使用 nop 解决数据相关

```
addiu $9, $9, 1
nop
nop
slt   $13, $8, $9
```

该方案的优点是硬件设计简单，不需要额外的数据相关检测或旁路电路。缺点是需要程序员或编译器手动安排空指令，程序长度变长，流水线利用率降低。若每个数据相关都通过 `nop` 解决，会产生大量无效周期，不适合本次希望优化流水线阻塞的目标。

本实验中只保留 MIPS 分支延迟槽中的必要 `nop`，不采用大量手工插入空指令的方式解决普通数据相关。

5.2 Wait 阻塞方案

Wait 阻塞方案由硬件在 ID 阶段检测数据相关。当当前指令的源寄存器 `rs` 或 `rt` 与后级流水线中尚未写回的目标寄存器相同时，ID 阶段暂停，等待数据可用后再继续执行。

本实验中，Wait 判断逻辑位于 `8_pipeline_cpu/decode.v`。修改后的判断逻辑如下：

Listing 22: decode.v 中 Wait 判断逻辑

```
assign rs_wait = src_rs & (rs!=5'd0)
               & ( (rs==EXE_wdest) ? ~rs_from_EXE :
                   (rs==MEM_wdest) ? ~rs_from_MEM :
                   (rs==WB_wdest ) ? ~rs_from_WB  : 1'b0 );

assign rt_wait = src_rt & (rt!=5'd0)
               & ( (rt==EXE_wdest) ? ~rt_from_EXE :
```

```
                (rt==MEM_wdest) ? ~rt_from_MEM :
                (rt==WB_wdest ) ? ~rt_from_WB  : 1'b0 );

assign ID_over = ID_valid & ~rs_wait & ~rt_wait
              & (~inst_jbr | IF_over);
```

这里的核心思想是：如果检测到相关，但后级结果尚不能旁路，则产生 `rs_wait` 或 `rt_wait`，使 `ID_over` 无效，从而阻塞流水线。

Wait 阻塞方案的优点是可以由硬件自动保证正确性，不需要程序员手动插入大量 `nop`。缺点是如果所有相关都等待到 WB 阶段，会导致流水线停顿过多，降低性能。因此本实验没有单独采用纯 Wait 方案，而是将其作为旁路无法解决时的保底机制。

5.3 旁路方案

旁路方案是将后级流水线中已经产生的结果直接送回前级使用，避免等待写回寄存器堆。本实验实现了 EXE、MEM、WB 到 ID 阶段的旁路网络。

在 `decode.v` 中，ID 阶段按照 EXE、MEM、WB 的优先级选择最新数据：

Listing 23: decode.v 中旁路数据选择

```
assign rs_data = rs_from_EXE ? EXE_fwd_data :
                 rs_from_MEM ? MEM_fwd_data :
                 rs_from_WB  ? WB_fwd_data  : rs_value;

assign rt_data = rt_from_EXE ? EXE_fwd_data :
                 rt_from_MEM ? MEM_fwd_data :
                 rt_from_WB  ? WB_fwd_data  : rt_value;
```

EXE 阶段对普通 ALU 结果产生旁路：

Listing 24: exe.v 中 EXE 级旁路

```
assign EXE_fwd_valid = EXE_valid & EXE_over & rf_wen
                     & (rf_wdest!=5'd0)
                     & ~mem_control[3] & ~mfhi & ~mflo & ~mfc0;

assign EXE_fwd_wdest = EXE_wdest;
assign EXE_fwd_data  = exe_result;
```

MEM 阶段在最终写回值已经可用时产生旁路：

Listing 25: mem.v 中 MEM 级旁路

```
assign MEM_fwd_valid = MEM_valid & MEM_over & rf_wen
                     & (rf_wdest!=5'd0)
                     & ~mfhi & ~mflo & ~mfc0;

assign MEM_fwd_wdest = MEM_wdest;
assign MEM_fwd_data  = mem_result;
```

WB 阶段则将本周期真正写入寄存器堆的数据直接旁路：

Listing 26: wb.v 中 WB 级旁路

```
assign WB_fwd_valid = rf_wen & (rf_wdest!=5'd0);
assign WB_fwd_wdest = rf_wdest;
assign WB_fwd_data  = rf_wdata;
```

旁路方案的优点是可以显著减少等待周期，提高流水线效率。缺点是需要增加旁路总线和选择逻辑，硬件复杂度高于单纯 Wait 阻塞。

5.4 本实验采用的冲突解决方案

本实验最终采用“旁路优先，必要时 Wait 阻塞”的方案。具体原则如下：

1. 普通 ALU 指令的结果若已经在 EXE、MEM 或 WB 阶段产生，则通过旁路送回 ID 阶段使用。
2. 如果当前相关结果尚未产生，例如 load 首拍、`mflo`、`mfhi`、`mfc0` 等情况，则保留 Wait 阻塞。
3. 分支和跳转指令保留 MIPS 延迟槽机制，延迟槽中使用 `nop` 保证程序行为明确。

自定义程序中，`addiu -> slt` 和 `slt -> beq` 可以通过旁路解决；而 `mflo -> addu` 中，`mflo` 的结果需要到后续阶段才可用，因此会出现必要的 Wait 阻塞。

5.5 典型数据相关验证

本实验通过 Vivado 波形验证了三类典型数据相关。

第一类是 `addiu -> slt` 数据相关。自定义程序中：

Listing 27: addiu 到 slt 的数据相关

```
addiu $9, $9, 1
slt   $13, $8, $9
```

`slt` 需要使用刚刚更新后的 `$9`。波形中可以观察到对应旁路信号有效，且 `rt_wait=0`，说明流水线没有因为该相关而等待到 WB 阶段。

第二类是 `slt -> beq` 数据相关：

Listing 28: slt 到 beq 的数据相关

```
slt   $13, $8, $9
beq   $13, $0, loop
```

`beq` 在 ID 阶段进行比较，需要使用 `slt` 刚产生的 `$13`。由于 `decode.v` 中分支比较也使用旁路后的 `rs_data` 和 `rt_data`，因此分支判断可以得到最新值：

Listing 29: decode.v 中分支比较使用旁路后的数据

```
assign rs_equql_rt = (rs_data == rt_data);
assign rs_ez       = ~(|rs_data);
assign rs_ltz      = rs_data[31];
```

第三类是 mflo -> addu 数据相关：

Listing 30: mflo 到 addu 的数据相关

```
1 mflo  $12
2 addu  $10, $10, $12
```

该相关不能直接在 EXE 阶段解决，因为 mflo 读取的是 LO 特殊寄存器，其结果需要等到后续阶段才能作为普通寄存器写回数据使用。因此波形中会出现 rt_wait=1，流水线暂停等待结果就绪。该现象说明本实验的方案不是盲目旁路，而是在结果尚未可用时保留必要阻塞，保证程序执行正确。

6 仿真验证

本节使用 Vivado 2017.1 对修改后的多周期 CPU 和五级流水线 CPU 进行行为仿真验证。仿真时主要观察各流水级 PC、当前指令、寄存器堆观察端口、数据存储器观察端口以及旁路和阻塞相关信号。

为了证明修改没有破坏原有程序，同时验证自定义程序和数据冲突处理，本节分别完成以下四组仿真：

1. 多周期 CPU 原始程序验证；
2. 五级流水线 CPU 原始程序验证；
3. 五级流水线 CPU 自定义程序验证；
4. 数据冲突处理效果验证。

6.1 多周期 CPU 原始程序验证

多周期 CPU 使用原始 COE 文件建立并初始化存储器 IP 核。仿真中主要观察 mem_addr、mem_data、rf_addr、rf_data 以及各阶段 PC 信号。

修复 IF_over 和 MEM_valid_r 的复位问题后，原始程序能够稳定执行。部分关键结果如下：

观察对象	期望结果	说明
Mem[0x14]	000c880d	原始程序写入数据存储器结果
Mem[0x1c]	00000011	原始程序写入数据存储器结果
$6	ffffffe2	原始程序寄存器运算结果

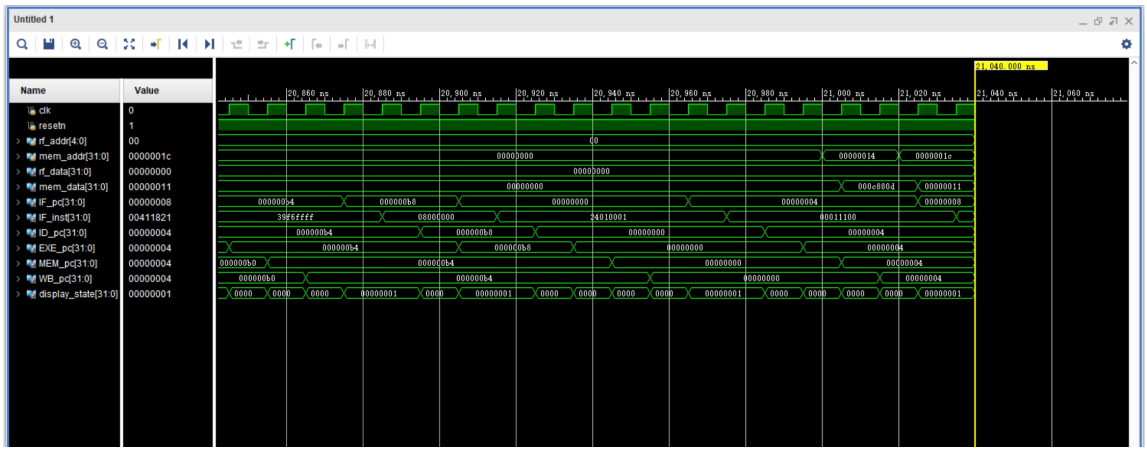


图 6.2: 多周期 CPU 原始程序仿真结果：Mem[0x14]=000c880d

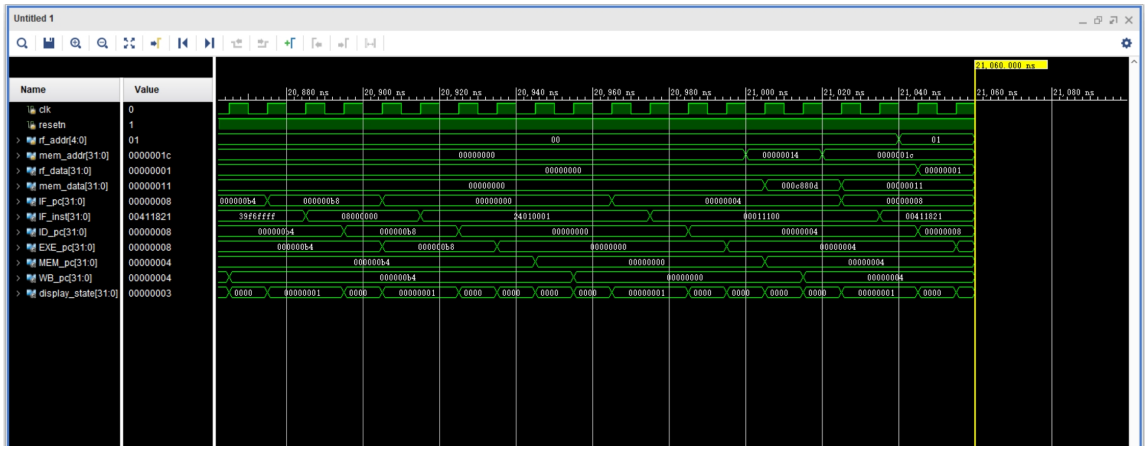


图 6.3: 多周期 CPU 原始程序仿真结果：Mem[0x1c]=00000011

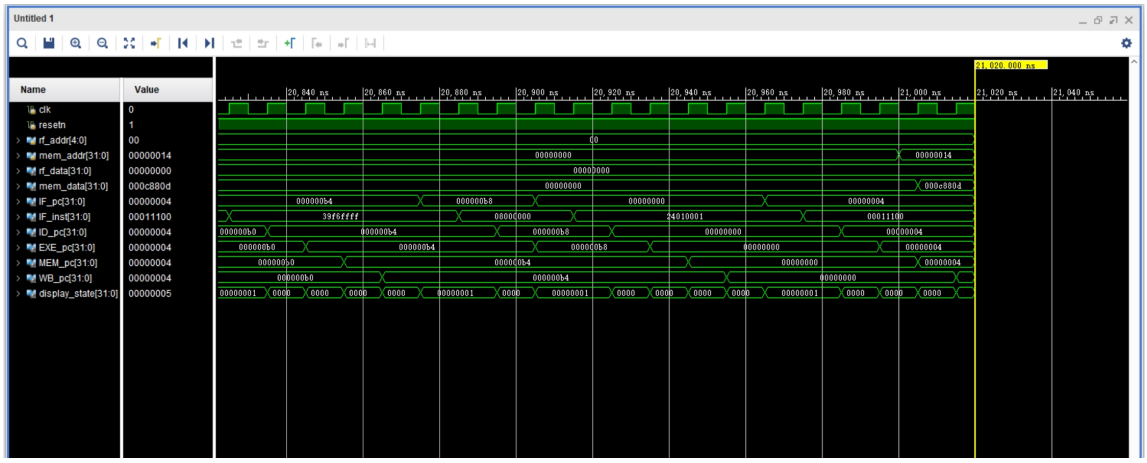


图 6.4: 多周期 CPU 原始程序仿真结果：寄存器 $6=fffffffe2

由上述波形可知，多周期 CPU 修复后能够正确执行原始测试程序，数据存储器和寄存器堆中的关键结果均符合预期。

6.2 五级流水线 CPU 原始程序验证

五级流水线 CPU 原始程序使用原始 COE 文件建立并初始化存储器 IP 核。该程序用于验证本次旁路优化和复位修复没有破坏原有 COE 程序。

仿真中观察 IF_pc、IF_inst、ID_pc、EXE_pc、MEM_pc、WB_pc、cpu_5_valid、mem_addr 和 mem_data 等信号。原始程序执行后的关键数据存储器结果如下：

观察对象	仿真结果
Mem[0x20]	12345678
Mem[0x24]	23456780
Mem[0x28]	34567800
Mem[0x3c]	01234567

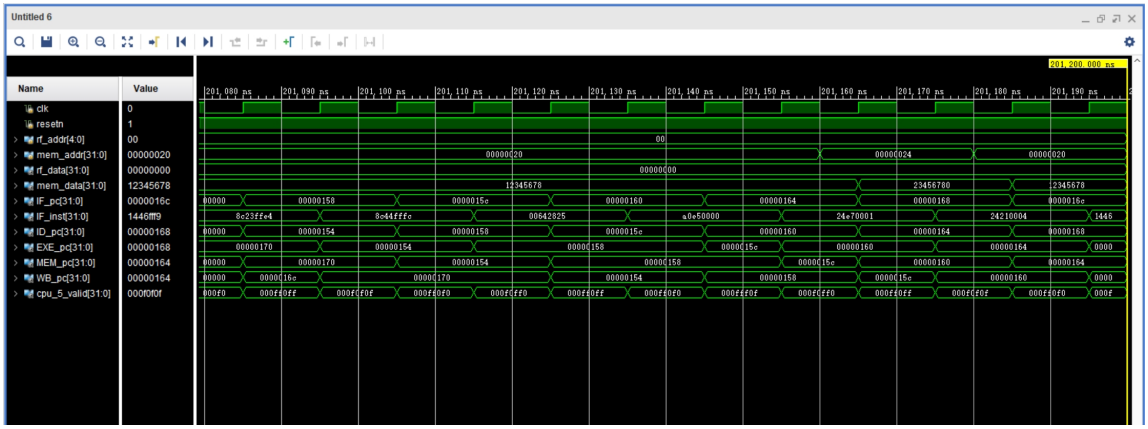


图 6.5: 五级流水线 CPU 原始程序仿真结果：Mem[0x20]=12345678

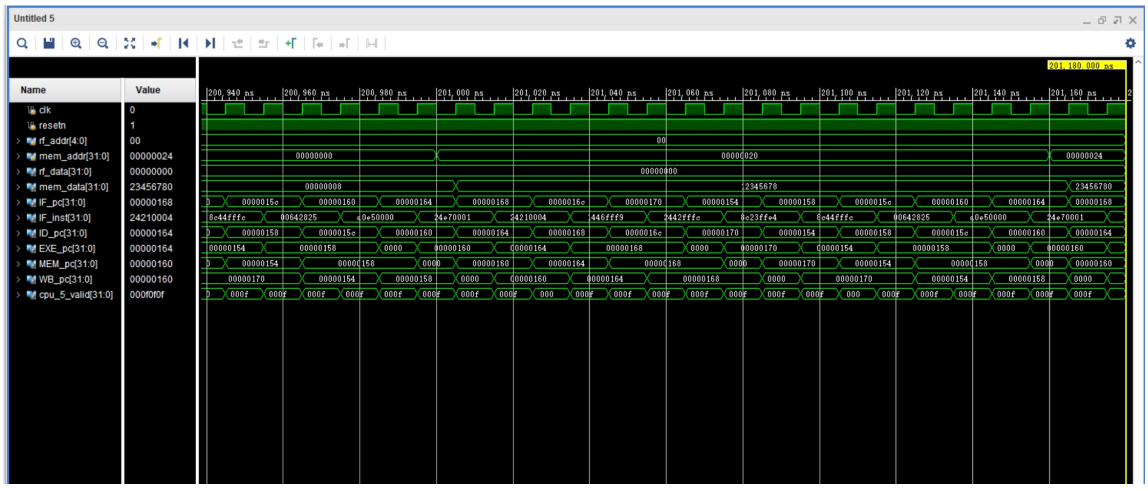


图 6.6: 五级流水线 CPU 原始程序仿真结果：Mem[0x24]=23456780

图 6.7: 五级流水线 CPU 原始程序仿真结果：Mem[0x28]=34567800

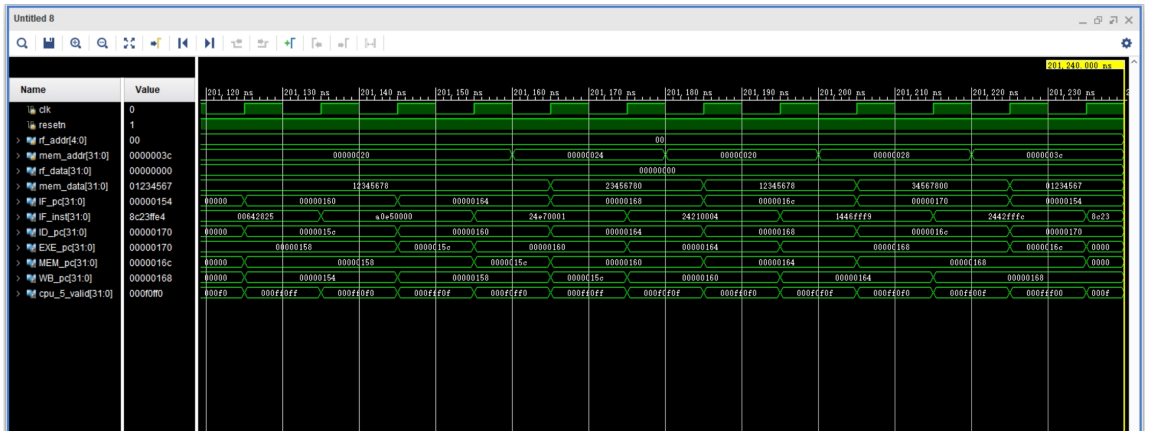


图 6.8: 五级流水线 CPU 原始程序仿真结果：Mem[0x3c]=01234567

由仿真结果可知，加入旁路和复位修复后，五级流水线 CPU 仍能正确执行原始程序，说明本次修改没有破坏已有 COE 程序。

6.3 五级流水线 CPU 自定义程序验证

运行自定义程序时，使用 pipeline_inst/inst_pipeline_custom.coe 建立并初始化自定义程序对应的指令 ROM IP 核，然后重新运行行为仿真。

自定义程序用于计算 1 到 5 的平方和、5!、二者之和以及比较结果。预期写入数据 RAM 的结果如下：

观察对象	含义	期望结果
Mem[0x80]	$1^2+2^2+3^2+4^2+5^2$	00000037
Mem[0x84]	$5!$	00000078
Mem[0x88]	$55+120$	000000af
Mem[0x8c]	$55<120$	00000001

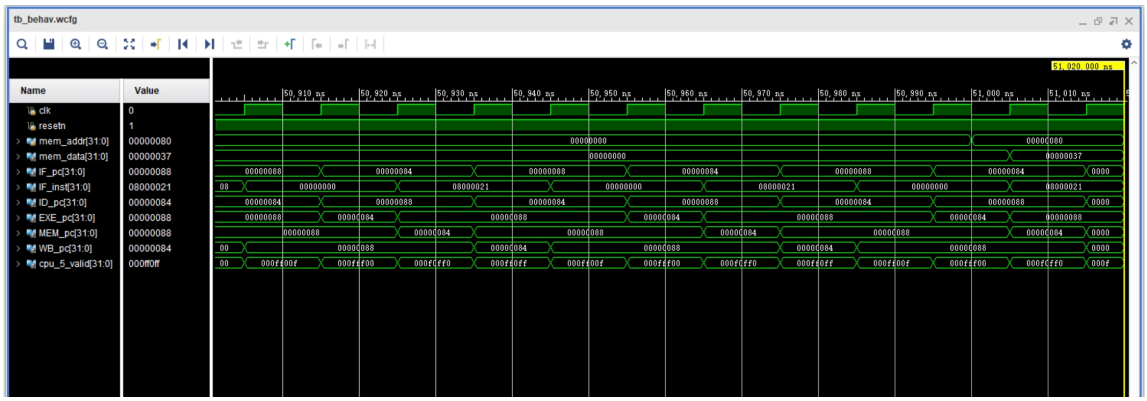


图 6.9: 自定义程序仿真结果：Mem[0x80]=00000037

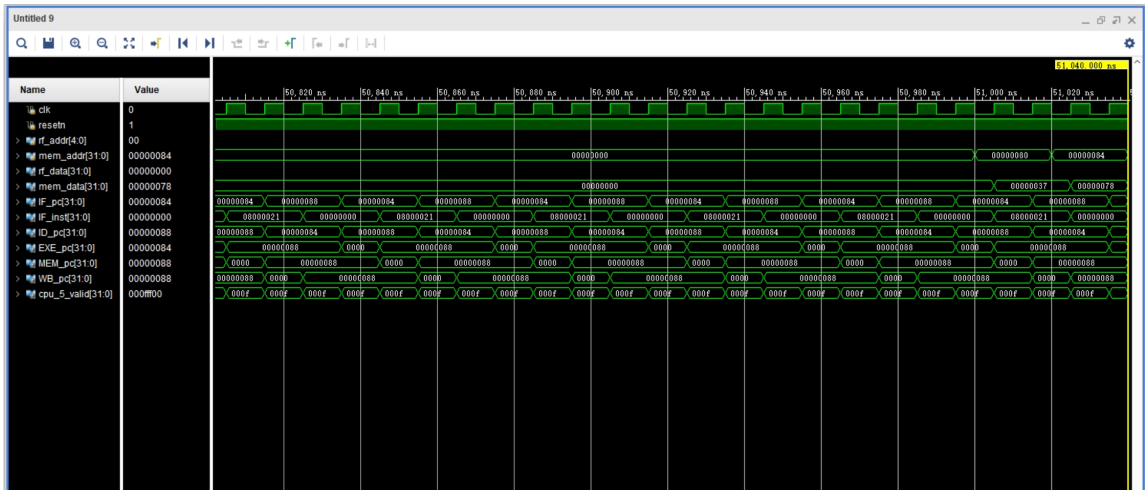


图 6.10: 自定义程序仿真结果：Mem[0x84]=00000078

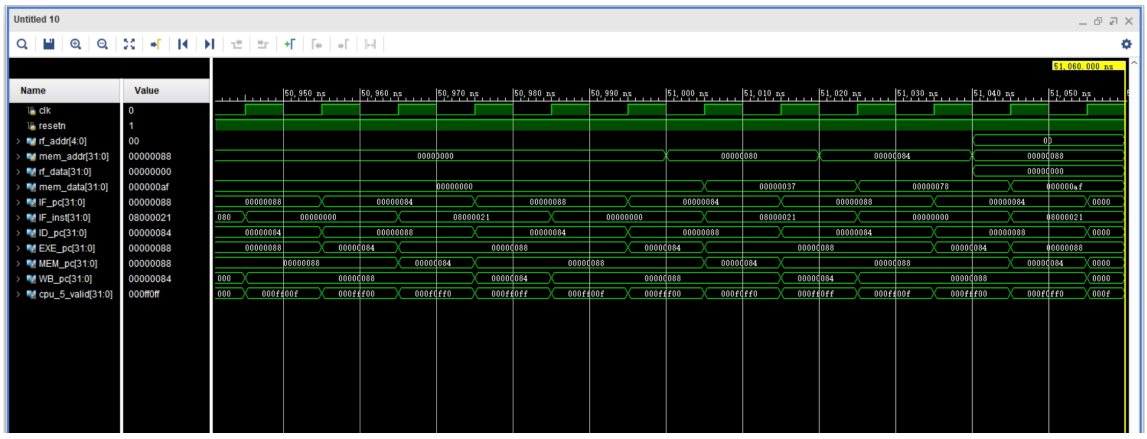


图 6.11: 自定义程序仿真结果：Mem[0x88]=000000af

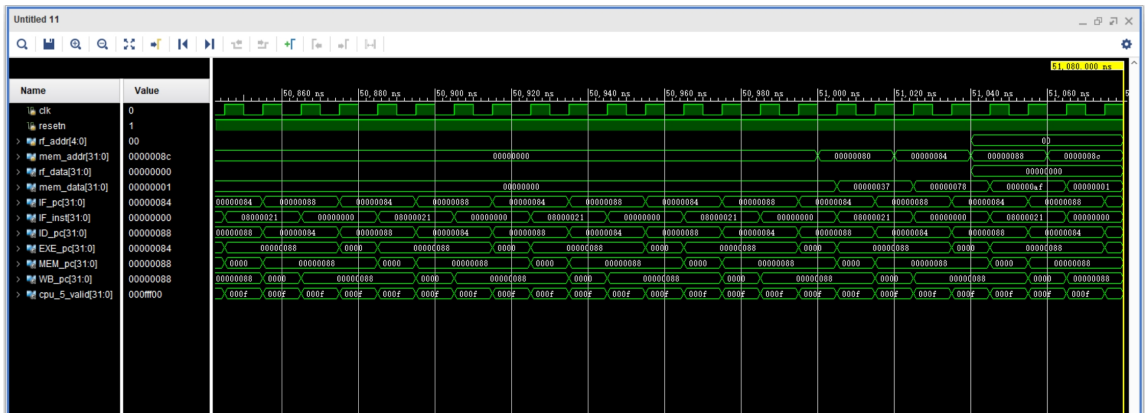


图 6.12: 自定义程序仿真结果：`Mem[0x8c]=00000001`

仿真结果与预期完全一致，说明自定义 MIPS 程序能够在修改后的五级流水线 CPU 上正确执行。

6.4 数据冲突处理效果验证

为了验证旁路和 Wait 阻塞机制，本次在自定义程序中保留了多种典型数据相关，并在 Vivado 波形中加入 `rs_wait`、`rt_wait`、`rs_data`、`rt_data`、`EXE_fwd_valid`、`MEM_fwd_valid`、`WB_fwd_valid` 等信号进行观察。

第一类是 `addiu -> slt` 相关：

Listing 31: addiu 到 slt 的数据相关

```
1 addiu $9, $9, 1
2 slt   $13, $8, $9
```

波形中可以看到相关寄存器数据通过旁路传递，且 `rt_wait=0`，说明该数据相关没有造成额外阻塞。

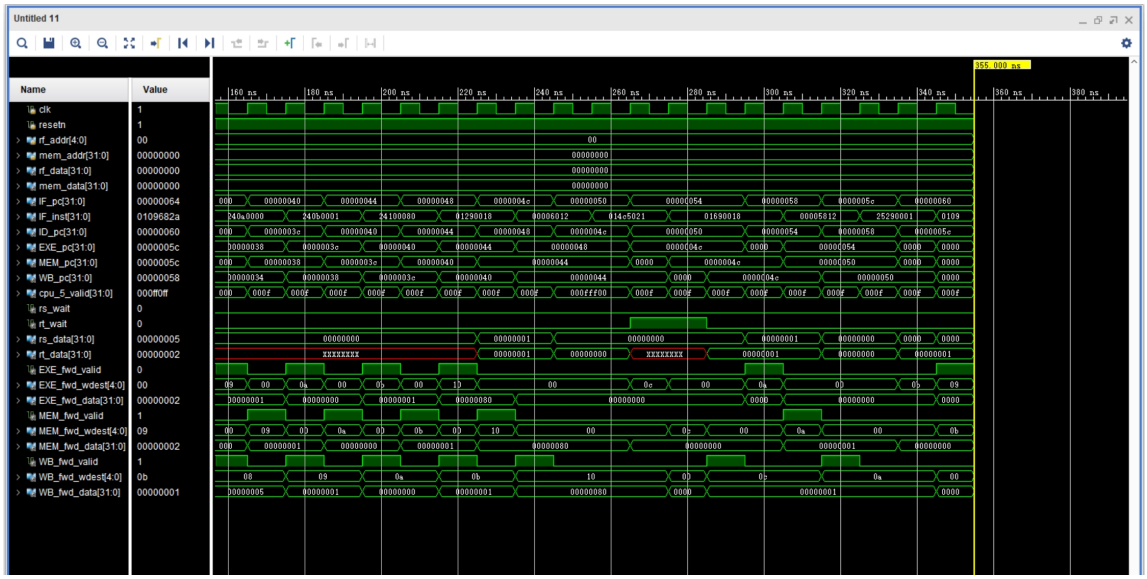


图 6.13: `addiu -> slt` 数据相关通过旁路解决

第二类是 `slt -> beq` 相关：

Listing 32: slt 到 beq 的数据相关

```
1 slt   $13, $8, $9
2 beq   $13, $0, loop
```

beq 在 ID 阶段进行比较，因此需要使用 slt 产生的最新 $13。由于本次修改后分支比较使用旁路后的 rs_data 和 rt_data，因此分支判断能够获得正确数据，波形中 rs_wait=0。

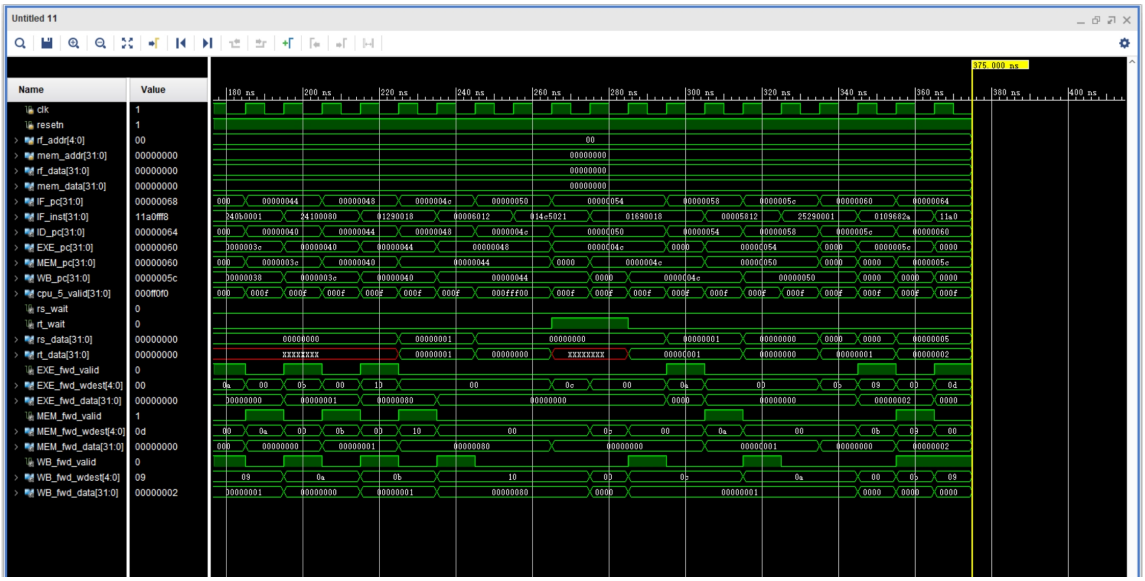


图 6.14: slt -> beq 数据相关通过旁路解决

第三类是 mflo -> addu 相关：

Listing 33: mflo 到 addu 的数据相关

```
1 mflo  $12
2 addu  $10, $10, $12
```

该相关不能在 EXE 阶段直接旁路，因为 mflo 的结果来自 LO 特殊寄存器，必须等结果真正可用后才能使用。因此波形中可以观察到 rt_wait=1，流水线进行必要阻塞。

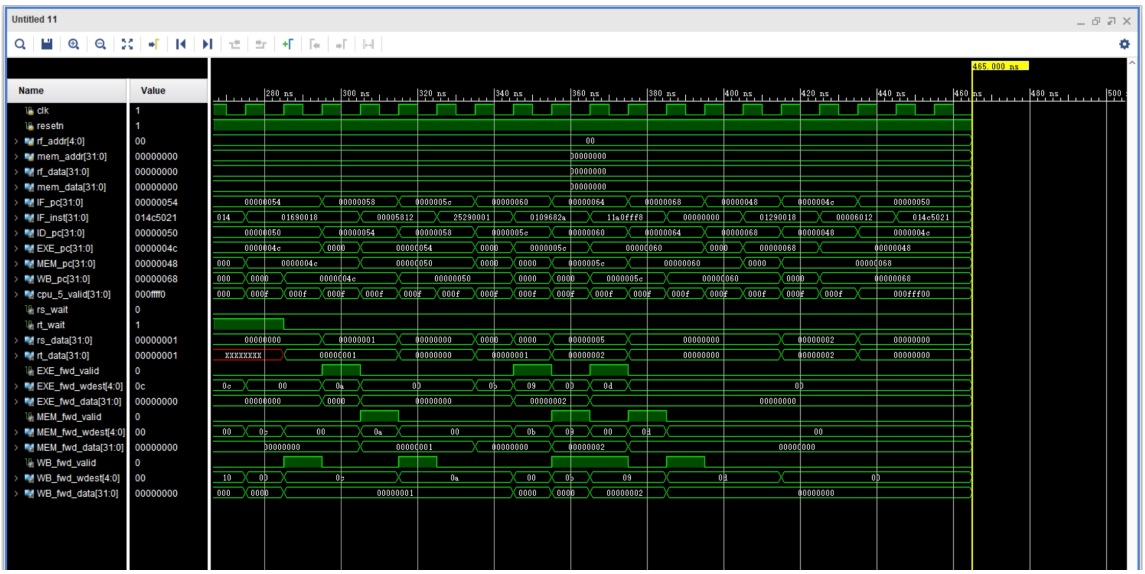


图 6.15: `mflo -> addu` 数据相关通过 Wait 阻塞保证正确性

综上，仿真结果表明：普通 ALU 相关可以通过旁路减少停顿，而结果尚未就绪的特殊相关仍会触发 Wait 阻塞。本实验采用的“旁路优先，必要时 Wait”的方案能够兼顾正确性和流水线效率。

7 上板验证

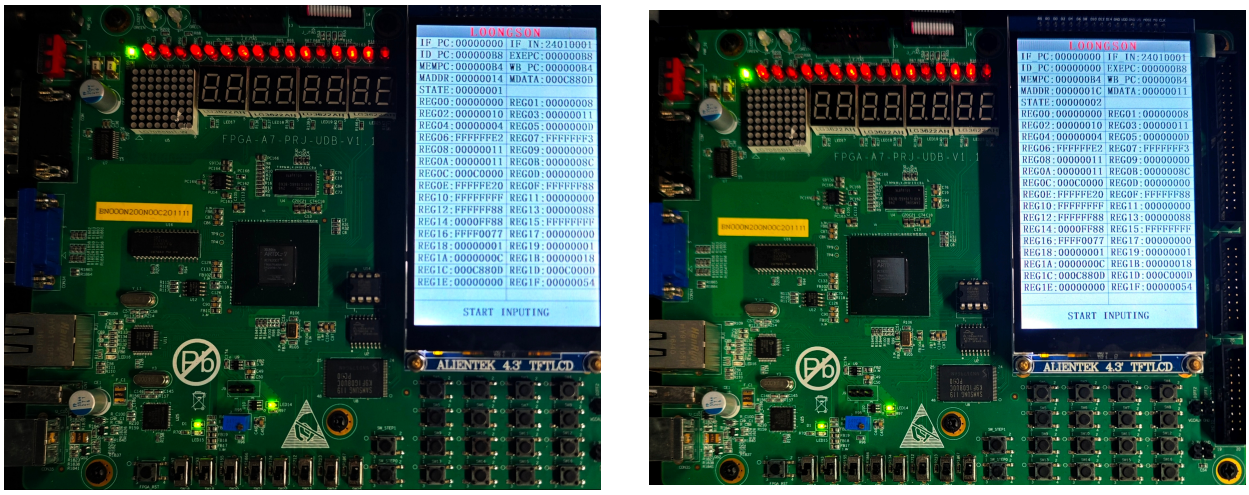
7.1 多周期 CPU 上板验证

多周期 CPU 上板验证使用工程 `project_88`，顶层模块为 `multi_cycle_cpu_display`。在 Vivado 中完成综合、实现和 bitstream 生成后，将程序下载到 FPGA 实验箱。下载完成后，通过 LCD 观察 CPU 当前执行状态、寄存器值以及数据存储器读出结果。

本实验重点验证修复后的多周期 CPU 是否能够正确执行原始测试程序。通过输入数据存储器地址并观察 `MDATA`，得到如下结果：

$$\texttt{Mem[0x14]} = \texttt{000c880d}, \qquad \texttt{Mem[0x1c]} = \texttt{00000011}$$

结果与仿真结果一致，说明多周期 CPU 在修复 `IF_over`、`MEM_valid_r` 以及顶层复位连接问题后，能够在实验箱上正确运行原始程序。



MADDR=0x14, MDATA=000c880d　　MADDR=0x1c, MDATA=00000011

图 7.16: 多周期 CPU 原始程序上板运行结果

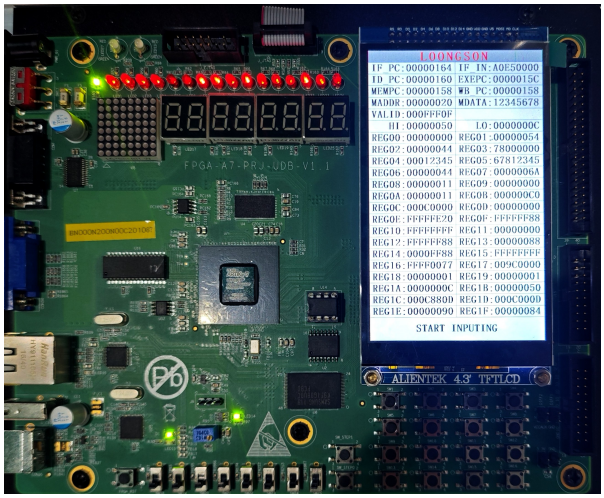
7.2 五级流水线 CPU 上板验证

五级流水线 CPU 上板验证使用工程 `project_89`，顶层模块为 `pipeline_cpu_display`。首先使用原始 COE 文件建立并初始化指令 ROM 和数据 RAM IP 核，重新生成 bitstream 后下载到 FPGA。下载后，通过 LCD 观察流水线各级 PC、当前指令、寄存器值以及数据存储器读出结果。

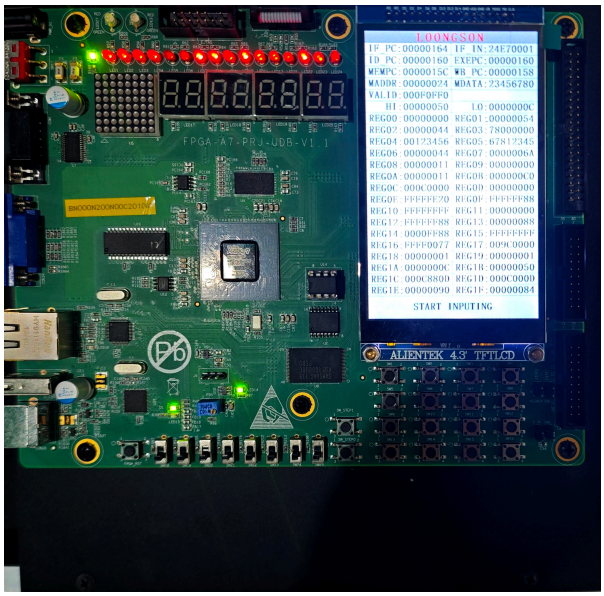
原始测试程序运行后，数据存储器中的关键结果如下：

$$\texttt{Mem[0x20]} = 12345678,$$
$$\texttt{Mem[0x24]} = 23456780,$$
$$\texttt{Mem[0x28]} = 34567800,$$
$$\texttt{Mem[0x3c]} = 01234567.$$

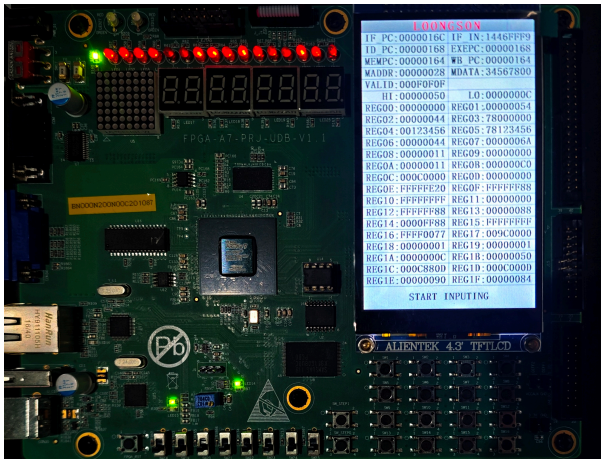
这些结果与 Vivado 仿真结果一致，说明修复后的五级流水线 CPU 能够正确执行原始测试程序，旁路和等待逻辑没有破坏原有程序的执行正确性。



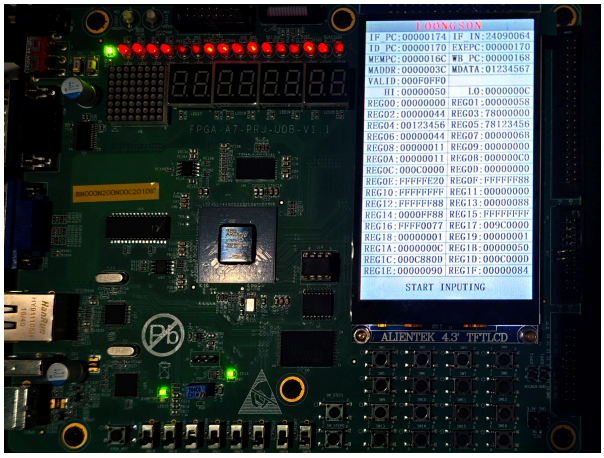
MADDR=0x20, MDATA=12345678



MADDR=0x24, MDATA=23456780



MADDR=0x28, MDATA=34567800



MADDR=0x3c, MDATA=01234567

图 7.17: 五级流水线 CPU 原始程序上板运行结果

7.3 自定义程序上板验证

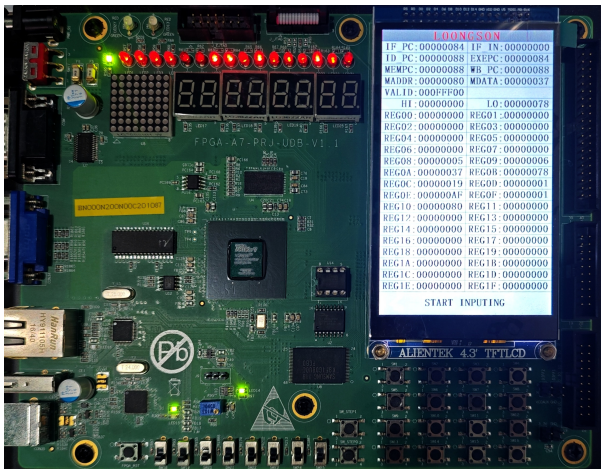
自定义程序上板验证时，先使用自定义程序对应的 `inst_pipeline_custom.coe` 重新建立并初始化指令 ROM IP 核，数据 RAM 继续使用按实验手册建立的 IP 核，然后在 Vivado 中重新生成 bitstream，并重新下载到 FPGA 实验箱。该程序完成一个具体的 MIPS 计算任务，并将关键结果写入数据存储器 0x80 至 0x8c 地址段。

自定义程序运行后的数据存储器结果如下：

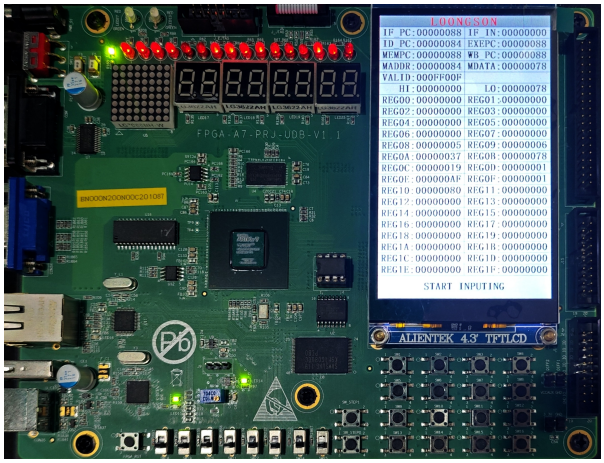
$$\begin{aligned}
\texttt{Mem[0x80]} &= \texttt{00000037},\\
\texttt{Mem[0x84]} &= \texttt{00000078},\\
\texttt{Mem[0x88]} &= \texttt{000000af},\\
\texttt{Mem[0x8c]} &= \texttt{00000001}.
\end{aligned}$$

其中 0x80、0x84 和 0x88 分别保存中间结果和最终计算结果，0x8c 保存判断结果。上板结果与仿真

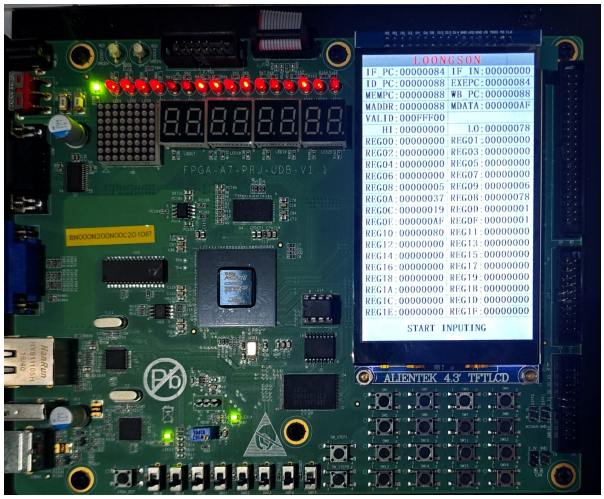
结果一致，说明自定义程序能够在加入旁路优化后的五级流水线 CPU 上正确运行，数据相关问题得到了正确处理。



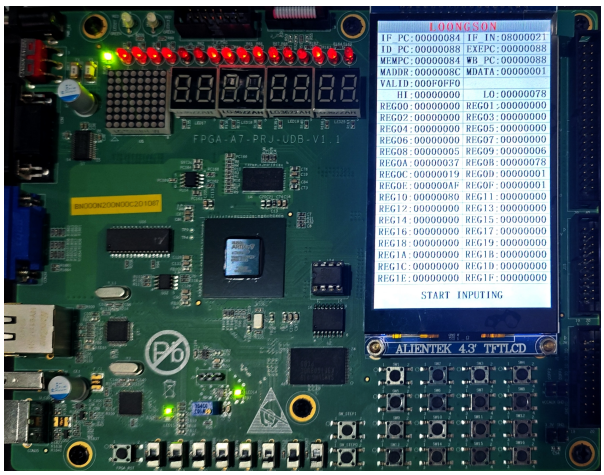
MADDR=0x80, MDATA=00000037



MADDR=0x84, MDATA=00000078



MADDR=0x88, MDATA=000000af



MADDR=0x8c, MDATA=00000001

图 7.18: 五级流水线 CPU 自定义程序上板运行结果

8 实验结果分析

8.1 Bug 修复效果分析

本次实验首先对多周期 CPU 和五级流水线 CPU 原始代码进行了复现和检查，发现原始设计中存在若干与复位、同步存储器等待和有效提交相关的问题。修改后，CPU 在仿真初始阶段的状态更加稳定，原始 COE 程序和自定义程序均能够正确执行。

对于多周期 CPU，主要修复了 `fetch.v` 中 `IF_over` 未复位和 `mem.v` 中 `MEM_valid_r` 未复位的问题。由于由 COE 建立的指令 ROM 和数据 RAM IP 核均为同步读，取指阶段和 load 访存阶段都需要等待一拍。修改前，相关等待寄存器初值可能为不确定值，导致状态机错误推进；修改后，等待寄存器在复位时清零，CPU 能够从确定状态开始执行。

对于五级流水线 CPU，修复内容包括 MEM 阶段 load 等待寄存器复位、乘法器内部状态复位、HI/LO 和 CP0 的有效提交保护等。修改后，流水线中的气泡、复位初始状态和被取消指令不会错误

修改体系结构状态，乘法指令第一次执行时也不会受到未知初值影响。

从仿真结果看,多周期 CPU 原始程序能够得到 `Mem[0x14]=000c880d`、`Mem[0x1c]=00000011`、寄存器 `$6=fffffe2` 等结果;五级流水线 CPU 原始程序能够得到 `Mem[0x20]=12345678`、`Mem[0x24]=23456780`、`Mem[0x28]=34567800`、`Mem[0x3c]=01234567` 等结果。说明修复后的 CPU 没有破坏原始测试程序功能。

8.2 旁路方案与 Wait 阻塞方案对比

原始五级流水线 CPU 主要依赖 Wait 阻塞解决数据相关。当 ID 阶段发现当前指令的源寄存器与后级流水线的目标寄存器相关时，流水线暂停等待，直到结果写回寄存器堆。该方案硬件逻辑相对简单，正确性较强，但会产生较多停顿周期。

本次实验在原有 Wait 机制基础上加入 EXE、MEM、WB 到 ID 阶段的旁路网络。对于普通 ALU 指令，结果在 EXE 或 MEM 阶段已经产生，不必等待 WB 写回即可被后续指令使用。因此，旁路方案能够减少不必要的流水线停顿，提高指令执行效率。

两种方案对比如下:

方案	实现方式	优点	缺点
Wait 阻塞	检测相关后暂停 ID 阶段	控制简单，正确性强	停顿周期多，效率较低
旁路	将后级结果提前送回 ID 阶段	减少停顿，提高流水线效率	需要增加旁路总线和选择逻辑

本实验最终采用“旁路优先，必要时 Wait”的折中方案。对于 `addiu -> slt`、`slt -> beq` 等普通数据相关，通过旁路获得最新结果，波形中对应 `rs_wait` 或 `rt_wait` 为 0；对于 `mflo -> addu` 这类结果尚未就绪的数据相关，仍然通过 Wait 阻塞保证正确性。该方案既避免了大量手工插入 `nop`，也比纯 Wait 阻塞具有更好的流水线效率。

8.3 自定义计算任务结果分析

本次自定义 MIPS 程序计算 1 到 5 的平方和、5!、二者之和以及大小比较结果。程序运行结束后，结果写入数据 RAM 的 `0x80` 到 `0x8c` 地址区间。

理论计算结果如下:

$$1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 55 = 0x37$$

$$5! = 1 \times 2 \times 3 \times 4 \times 5 = 120 = 0x78$$

$$55 + 120 = 175 = 0xAF$$

$$55 < 120$$

因此，程序预期写入结果为:

地址	含义	期望结果
0x80	平方和	00000037
0x84	阶乘	00000078
0x88	平方和与阶乘之和	000000af
0x8c	比较结果	00000001

Vivado 仿真中观察到的结果与理论值一致，说明自定义汇编程序、机器码生成、COE 文件建立并初始化存储器 IP 核以及流水线 CPU 执行过程均正确。同时，该程序中包含乘法、`mflo`、循环分支、store 写存储器和多种 RAW 数据相关，可以有效验证本次旁路和 Wait 阻塞机制。

9 实验总结

本次实验在复现实验指导手册中多周期 CPU 和五级流水线 CPU 的基础上，对原始代码进行了 bug 定位、修复和优化。

在多周期 CPU 部分，修复了取指阶段 `IF_over` 和访存阶段 `MEM_valid_r` 未复位的问题，使同步 ROM/RAM 的等待控制从确定状态开始，保证原始程序稳定执行。

在五级流水线 CPU 部分，修复了 MEM 阶段 load 等待寄存器复位、乘法器内部状态复位、HI/LO 与 CP0 有效提交保护等问题，并实现了 EXE、MEM、WB 到 ID 阶段的旁路网络。修改后，普通数据相关可以通过旁路减少停顿，结果尚未就绪的特殊相关仍通过 Wait 阻塞保证正确性。

此外，本次仿照单周期 CPU 实验，自行编写了一个完整的 MIPS 计算任务，完成平方和、阶乘、求和和比较运算，并生成对应 COE 文件建立指令 ROM IP 核后交由五级流水线 CPU 执行。仿真结果表明，数据 RAM 中的最终结果与理论计算完全一致。

综上，本次实验完成了原始实验复现、bug 修复、自定义程序设计、数据冲突处理和仿真验证，覆盖了基本要求和提高要求。

附录 A 关键代码片段

A.1 多周期 CPU 关键修改

Listing 34: 多周期 CPU 取指阶段复位修复

```
always @(posedge clk)
begin
    if (!resetn)
        IF_over <= 1'b0;
    else
        IF_over <= IF_valid;
end
```

Listing 35: 多周期 CPU 访存阶段 Load 等待修复

```
reg MEM_valid_r;
always @(posedge clk)
begin
    if (!resetn)
```

```
        MEM_valid_r <= 1'b0;
    else
        MEM_valid_r <= MEM_valid;
end

assign MEM_over = inst_load ? MEM_valid_r : MEM_valid;
```

A.2 五级流水线 CPU 关键修改

Listing 36: ID 阶段旁路数据选择

```
assign rs_data = rs_from_EXE ? EXE_fwd_data :
                 rs_from_MEM ? MEM_fwd_data :
                 rs_from_WB  ? WB_fwd_data  : rs_value;

assign rt_data = rt_from_EXE ? EXE_fwd_data :
                 rt_from_MEM ? MEM_fwd_data :
                 rt_from_WB  ? WB_fwd_data  : rt_value;
```

Listing 37: ID 阶段 Wait 判断

```
assign rs_wait = src_rs & (rs!=5'd0)
              & ( (rs==EXE_wdest) ? ~rs_from_EXE :
                  (rs==MEM_wdest) ? ~rs_from_MEM :
                  (rs==WB_wdest ) ? ~rs_from_WB  : 1'b0 );

assign rt_wait = src_rt & (rt!=5'd0)
              & ( (rt==EXE_wdest) ? ~rt_from_EXE :
                  (rt==MEM_wdest) ? ~rt_from_MEM :
                  (rt==WB_wdest ) ? ~rt_from_WB  : 1'b0 );
```

Listing 38: EXE/MEM/WB 级旁路输出

```
assign EXE_fwd_valid = EXE_valid & EXE_over & rf_wen
                     & (rf_wdest!=5'd0)
                     & ~mem_control[3] & ~mfhi & ~mflo & ~mfc0;

assign MEM_fwd_valid = MEM_valid & MEM_over & rf_wen
                     & (rf_wdest!=5'd0)
                     & ~mfhi & ~mflo & ~mfc0;

assign WB_fwd_valid = rf_wen & (rf_wdest!=5'd0);
```

Listing 39: HI/LO 有效提交保护

```
if (!resetn)
    hi <= 32'd0;
else if (WB_over && hi_write)
```

```
4       hi <= mem_result;
5
6   if (!resetn)
7       lo <= 32'd0;
8   else if (WB_over && lo_write)
9       lo <= lo_result;
```

附录 B 自定义 MIPS 汇编程序

Listing 40: 自定义 MIPS 汇编程序

```
1   addiu $8,  $0, 5
2   addiu $9,  $0, 1
3   addiu $10, $0, 0
4   addiu $11, $0, 1
5   addiu $16, $0, 0x80
6
7   loop:
8   mult  $9, $9
9   mflo  $12
10  addu  $10, $10, $12
11  mult  $11, $9
12  mflo  $11
13  addiu $9, $9, 1
14  slt   $13, $8, $9
15  beq   $13, $0, loop
16  nop
17
18  sw    $10, 0($16)
19  sw    $11, 4($16)
20  addu  $14, $10, $11
21  sw    $14, 8($16)
22  slt   $15, $10, $11
23  sw    $15, 12($16)
24
25  end:
26  j end
27  nop
```

附录 C 自定义 COE 文件内容

Listing 41: 自定义程序 COE 文件内容

```
1   memory_initialization_radix=16;
2   memory_initialization_vector=
3   00000000,
```

```
00000000,
00000000,
00000000,
00000000,
00000000,
00000000,
00000000,
00000000,
00000000,
00000000,
00000000,
00000000,
00000000,
24080005,
24090001,
240a0000,
240b0001,
24100080,
01290018,
00006012,
014c5021,
01690018,
00005812,
25290001,
0109682a,
11a0fff8,
00000000,
ae0a0000,
ae0b0004,
014b7021,
ae0e0008,
014b782a,
ae0f000c,
08000021,
00000000;
```